

N.E.D.S (Smart Lathi) — Cybersecurity Implementation Plan

VENJEX

Smart Lathi is secured end to end, from the sensor to the RPF website: every alert is signed and hash-chained at the moment it is made, travels only over authenticated and encrypted links, and can be added or read but never edited. This keeps detections trustworthy for officers and admissible as evidence in NDPS cases, even when the lathi works fully offline.

1. Purpose and scope

Smart Lathi's alerts decide whether an RPF officer opens a passenger's bag, and its GPS + timestamped logs are meant to support NDPS cases. So security here has three jobs: an alert cannot be faked or silenced, a log cannot be edited after the fact, and the device cannot be taken over. Our problem statement sits under the **Blockchain & Cybersecurity** theme, so this plan is part of the solution, not an add-on.

Security goals

Goal	What it means for Smart Lathi
Integrity of detection	Sensor readings and the Model A/B verdict reach the officer unchanged
Integrity of evidence	Every alert log (reading, verdict, GPS, time, officer) is tamper-evident and traceable
Availability	SCAN → alert works offline, even if Wi-Fi is jammed or the phone is lost
Confidentiality	Logs, locations and officer identity are unreadable if the lathi is stolen
Authenticity	Only RPF-issued phones can see the dashboard; only team-signed firmware and models can run

In scope: all five layers from the architecture slide (L1 Intake → L5 Alert), location & logs, power/build, the ML training pipeline, and the connection that sends live updates from the lathi to our website.

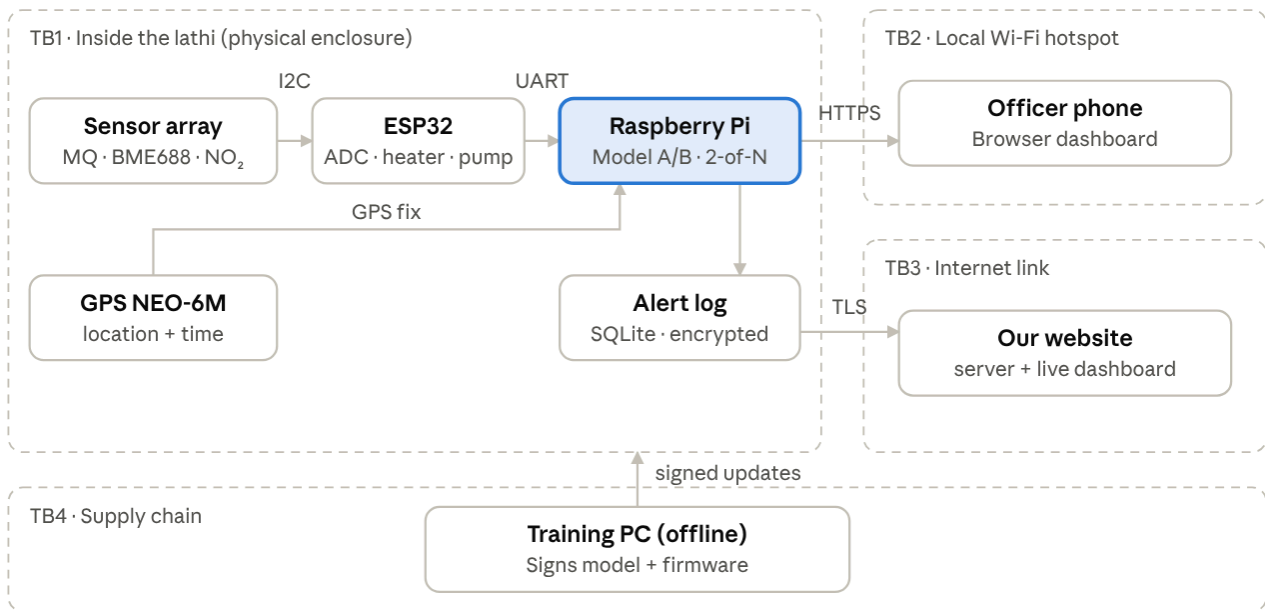
Stage: for this clearance round we present sensor-level detection with our ML models. The security architecture in this document is our implementation plan for the showcase and beyond; section 8 shows what we will implement by the showcase and what follows in an RPF pilot.

The AI brain is a Raspberry Pi 4; every Pi control in this document is written for it.

2. System overview and trust boundaries

Smart Lathi has four trust boundaries, and every security control in this plan sits on one of them. Data is born inside the lathi, crosses to the officer's phone over the Pi hotspot, and crosses the internet to update our website; code and models enter from the team's training PC.

Smart Lathi architecture and trust boundaries



Smart Lathi architecture · 4 trust boundaries

Dashed boxes are trust boundaries. The Raspberry Pi is highlighted because it holds the model, the keys and the evidence log, so it is the single most valuable target.

Boundary	What crosses it	Main risk	Controls in
TB1 · Physical enclosure	Sensor signals, UART frames, microSD card, debug ports	Lathi stolen or opened; SD card read; ESP32 or sensors swapped	4.1, 4.2, 4.3, 4.6

Boundary	What crosses it	Main risk	Controls in
TB2 · Local Wi-Fi hotspot	Dashboard pages, alerts, officer login	Unknown phone joins; traffic sniffed; fake hotspot; Wi-Fi jammed	4.5
TB3 · Internet link to our website	Live alerts, logs, device health, website logins	Updates intercepted or replayed; fake device posts alerts; website hacked	4.8, 4.9
TB4 · Supply chain	ESP32 firmware, model file, Python packages	Poisoned model or malicious firmware installed	4.4, 4.7

3. Threat model

The biggest risks are an edited evidence log and a silenced alert, because either one can let a trafficker walk or wreck an NDPS case in court. We model threats with STRIDE across the four boundaries.

What we protect (assets)

Asset	Where it lives	Why it matters
Alert log (reading, verdict, GPS, time, officer)	Pi, SQLite	Evidence for NDPS cases; must stand up to a defence lawyer
Detection verdict in transit	ESP32 → Pi → phone	Decides whether a bag is opened
Model A / Model B files	Pi storage	Tampering can hide one drug class or flood false alarms
Device keys and officer credentials	Pi	Unlock everything else
GPS history	Pi, website database	Reveals patrol routes and hotspots
ESP32 firmware	ESP32 flash	Controls heater, pump and raw readings

STRIDE analysis

Threat	Example on Smart Lathi	Planned control
Spoofing	Unknown phone opens the dashboard; fake ESP32 feeds readings; fake lathi posts alerts to the website; stolen website login	WPA2/WPA3 + officer PIN login; authenticated UART frames; per-device certificate
Tampering	Alert row deleted from SQLite or the website database; model swapped to ignore heroin; firmware changed	Hash-chained, signed log; signed model checked at boot; ESP32 Secure Boot
Repudiation	Officer says "I never scanned that bag"	Each entry carries officer ID, device ID, time and signature
Information disclosure	Stolen SD card exposes GPS trail and cases	Encrypted storage; keys kept off the plain SD card
Denial of service	Wi-Fi deauth jamming; battery drain; UART flood	Buzzer/LED alert works without Wi-Fi; watchdogs; rate limits
Elevation of privilege	Attacker on hotspot gets a shell via default password or Flask debug mode	No SSH on hotspot; non-root service; firewall; debug off

Who we defend against

- **Trafficker on the train:** wants no alert, so may jam Wi-Fi or trigger false alarms to make officers distrust the lathi.
- **Nearby attacker on the platform:** a laptop or phone that joins the hotspot, sniffs traffic, or sets up a fake hotspot with the same name.
- **Internet attacker:** targets our public website with stolen passwords, SQL injection or XSS, or posts fake alerts to the device API to flood it or plant false records.
- **Thief or finder of a lost lathi:** pulls the microSD card or plugs into the ESP32's USB port.
- **Insider:** someone with access who deletes an inconvenient alert or plants a fake one against a passenger.
- **Supply-chain attacker:** a tampered Python package, model file or firmware image.

4. Where and how each control is implemented

Every control below is mapped to a layer of the Smart Lathi architecture (L1 Intake → L5 Alert) and to the part of the device that runs it. Each subsection says where the control lives, how it works, and the tool we plan to use.

4.1 Sensor chamber and ESP32 edge controller (L1-L3)

Where: ESP32 on the soldered PCB in the electronics neck, plus the ADS1115 ADC and MOSFET drivers.

How it works

- **Secure Boot v2:** the ESP32's ROM bootloader checks a signature on every firmware image, so only team-signed firmware can run. A swapped or reflashed ESP32 simply won't boot.
- **Flash encryption:** firmware and stored calibration values are encrypted on flash, so pulling the chip reveals nothing useful.
- **Lock debug ports:** disable JTAG and put UART download mode into secure mode through eFuses on production units. eFuse changes are permanent, so we do this only on final builds, never on dev boards.
- **Sensor sanity checks:** ESP32 rejects readings outside each sensor's physical range and flags stuck or zero values. A cut wire or dead sensor reports **SENSOR FAULT**, never a silent **CLEAR**.
- **Hard safety limits in firmware:** heater PWM duty and the 3–5 s burst length are capped inside the ESP32 itself. Even a hacked Pi cannot command the ~180 °C mesh to stay on and melt the housing.
- **Physical tamper evidence:** security screws and tamper-evident seals on the neck and handle; opening them is visible at inspection.

Tools: ESP-IDF / PlatformIO with `espsecure` and `espefuse`.

4.2 ESP32 → Pi UART link

Where: the wired UART running inside the lathi shaft.

How it works

- Every packet is a frame: device ID, sequence number, timestamp, sensor values, CRC and an **HMAC-SHA256** tag.
- The shared HMAC key is written once at build time into ESP32 encrypted storage and into the Pi's protected key file.

- The Pi drops any frame with a bad tag, a repeated sequence number (replay) or a time gap that doesn't fit.
- Commands from the Pi to the ESP32 (start scan, heater on, pump on) are also tagged and checked against a short allowlist.

Tools: mbedTLS on ESP32; Python `hmac` / `hashlib` on the Pi.

4.3 Raspberry Pi hardening (L4 AI brain)

Where: the Raspberry Pi operating system and the detection service.

How it works

- **Minimal OS:** field units run Raspberry Pi OS Lite (no desktop). The desktop image stays on the dev bench only.
- **Accounts:** remove default credentials; one named admin user with a strong password; no auto-login.
- **Remote access:** SSH off in the field. For maintenance, key-only SSH over a USB cable, never over the hotspot.
- **Firewall:** nftables/ufw allows only the dashboard's HTTPS port on the hotspot interface and drops everything else. Bluetooth and unused interfaces are disabled.
- **Least privilege:** the detection and dashboard apps run as a non-root systemd service with `NoNewPrivileges` and a read-only view of the system.
- **Read-only root filesystem:** the overlay file system is enabled, so malware cannot survive a reboot. Only the encrypted data partition is writable.
- **Watchdog:** the Pi's hardware watchdog restarts the detection service if it hangs, so the lathi never sits frozen in "ready".
- **Patching:** OS and Python packages updated at each maintenance cycle, with versions pinned.
- **Signed boot:** the Pi 4 supports Raspberry Pi's secure-boot provisioning, so its bootloader loads only a boot image signed with our key. Like the ESP32 eFuses, this writes permanent settings into the chip, so we enable it at the RPF pilot; until then the signature checks in 4.4 and 4.7 protect the model and software.

4.4 AI model and training pipeline

Where: the offline training PC and the model files loaded on the Pi.

How it works

- **Offline training:** Model A (narcotics) and Model B (explosives) are trained on a PC that doesn't connect to the lathi.

- **Hash and sign:** every release records the SHA-256 of the training dataset and the model, and the model is signed with an Ed25519 key kept offline.
- **Verify before load:** the Pi checks the signature before loading. A bad signature means the file is rejected and the last good model is kept.
- **Pickle risk:** scikit-learn models saved with pickle/joblib can run code when loaded, so the signature check must come first. We will also test exporting to ONNX to remove this risk entirely.
- **Model version in every log:** each alert records which model hash made the decision, so any verdict can be replayed later.
- **Out-of-range inputs:** readings far outside the training range return **UNCERTAIN – RESCAN** instead of a confident verdict, on top of the 2-of-N rule.
- **Dependencies:** `pip install --require-hashes` from a pinned requirements file.
- **Data poisoning:** only reviewed proxy-trial data enters the training set, with each trial session logged.

4.5 Wi-Fi hotspot and phone dashboard (L5 Alert)

Where: the Pi's Wi-Fi hotspot and the Flask/FastAPI dashboard opened on the officer's phone.

How it works

- **Hotspot:** WPA2 (WPA3 where the phone supports it) with a unique, long passphrase per lathi, and a cap of 1-2 connected devices.
- **HTTPS only:** the dashboard is served over HTTPS with a certificate unique to each lathi. The officer's phone trusts that one certificate at issue time, which also exposes a fake hotspot pretending to be the lathi.
- **Officer login:** PIN or password stored as a bcrypt/Argon2 hash; sessions expire after one shift; lockout after 5 wrong attempts.
- **Roles:** Officer (view and acknowledge alerts), Supervisor (export logs), Maintenance (updates). No role can delete a log entry from the UI.
- **Web hygiene:** debug mode off, CSRF tokens on every form, Jinja2 auto-escaping for EN/Hindi/Assamese text, strict security headers, input checks on every API field.
- **Alert without Wi-Fi:** buzzer + LED fire from the lathi itself. If the hotspot is jammed, the officer still gets the alert and the log is still written; the phone catches up on reconnect.

Tools: hostapd, Flask/FastAPI, bcrypt/argon2-cffi, Flask-WTF.

4.6 Evidence logs and data at rest (Location & Logs)

Where: the SQLite alert log and GPS data on the Pi's encrypted data partition.

How it works

- **Hash-chained ledger:** each entry stores the SHA-256 hash of the entry before it. Deleting or editing any past row breaks the chain and shows up at once. This is the blockchain idea from our theme, run on-device without needing a network.
- **Signed entries:** each entry is signed with the lathi's own Ed25519 key, proving which device wrote it.
- **What one entry holds:** entry ID, device ID, officer ID, GPS time, latitude/longitude + fix quality, sensor-reading summary, model hash, verdict, which sensors agreed (2-of-N), previous hash, signature.
- **Trusted time:** GPS time is used because the Pi has no battery clock; an entry made without a GPS fix is marked as such.
- **Append-only:** the app has no delete or edit function for alerts.
- **Encryption at rest:** the database is encrypted with SQLCipher (AES-256). Phase 1 keeps the key in a root-only file; Phase 2 moves keys into an ATECC608-class secure element on I2C so they can't be copied off the SD card.
- **Court-ready export:** a Supervisor exports a signed bundle (entries + signatures + final chain hash) with a small verification script, so a forensic lab can check it independently. The export itself is logged, forming a chain of custody.

Tools: Python `hashlib`, PyNaCl (Ed25519), SQLCipher.

4.7 Firmware, model and app updates

Where: the maintenance workflow, from the training PC to the lathi.

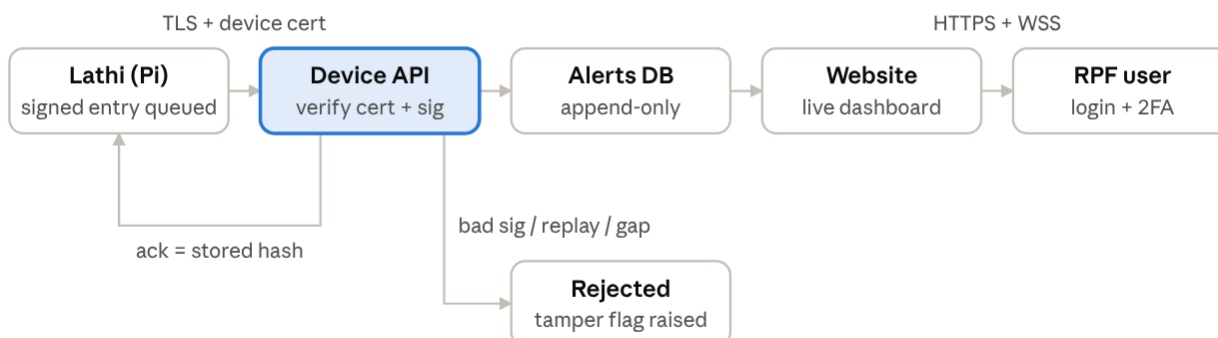
How it works

- **Phase 1:** updates only over a USB cable at the RPF post; no over-the-air updates through the hotspot.
- **Signed update package:** ESP32 firmware, model files and Pi app are bundled and signed; the lathi verifies before installing.
- **Anti-rollback:** version numbers only go up, so an attacker can't reinstall an older vulnerable build. The previous good version is kept for recovery.
- **Key custody:** signing keys stay offline on a dedicated machine; a release needs two team members to approve.
- **Phase 2 OTA:** ESP32 uses signed HTTPS OTA with the anti-rollback eFuse; the Pi pulls signed packages from our website's server over mutual TLS (4.8).

4.8 Lathi → website connection (live updates)

The lathi pushes every new log entry to our website as soon as it has internet, over an encrypted channel where both sides prove who they are. The website stores only entries it can verify came from a registered lathi.

Lathi to website: live update path



Lathi to website · verify before store, ack before delete

The Device API is the security gate: anything that fails a check is rejected and flagged, and the lathi keeps the entry until the website confirms it stored exactly that entry.

Where: the Pi's upload service, the internet link, and the website's Device API.

How it works

- **Getting online:** the Pi reaches the internet through a 4G USB dongle or RPF post Wi-Fi (to be finalised). The officer's hotspot stays a separate network, so the phone never bridges the lathi to the internet.
- **Encrypted channel:** TLS 1.3 for everything — MQTT over TLS (port 8883) for live updates, or HTTPS for a REST API. Plain HTTP and plain MQTT are switched off on the server.
- **Device identity (mutual TLS):** each lathi has its own certificate. The server refuses any connection without a valid, non-revoked certificate, so a laptop can't pretend to be a lathi.
- **Server identity (pinning):** the lathi only talks to our website's certificate, so a fake server can't collect the data.
- **Signed payloads:** the lathi sends the signed log entry from 4.6, not loose values. Even if someone got past TLS, a changed entry fails the Ed25519 signature check.
- **Replay and gap checks:** each entry carries its sequence number and previous hash. The server rejects duplicates and out-of-order entries and flags missing ones.
- **Acknowledgement:** the server replies with the hash of what it stored. The lathi marks the entry as synced only when that hash matches; otherwise it retries.

- **Store-and-forward:** with no internet, entries wait in the encrypted database and upload in order later, so offline patrols lose nothing.
- **One-way trust:** the website can only send back acks, a health ping or a revocation notice. It can never trigger a scan, change a verdict or delete anything on the lathi.
- **Rate limits:** each device has an upload limit, so a stolen certificate can't flood the website.
- **Revocation:** a lost lathi's certificate is revoked on the website and it can no longer connect.

Tools: Mosquitto (MQTT broker) or FastAPI, paho-mqtt on the Pi, OpenSSL or step-ca to issue device certificates.

4.9 Website security

The website is the only part of Smart Lathi on the public internet, so it gets standard web-app hardening plus one extra rule: evidence can be added and read, never edited.

Where: the web server, the database behind it, and the admin pages.

How it works

- **HTTPS everywhere:** TLS 1.3 with HSTS; a free Let's Encrypt certificate for the showcase, a government-approved cloud for a pilot. Only ports 443 (website) and 8883 (device MQTT) are open.
- **Two separate doors:** the Device API accepts certificates and can only add entries; the user site accepts logins and can only read and acknowledge. A user account can never post an alert, and a device can never read other devices' data.
- **Login:** passwords hashed with Argon2/bcrypt; two-factor login (authenticator app) for Supervisor and Admin; lockout after repeated failures; sessions time out.
- **Roles:** Officer sees alerts for their post; Supervisor sees the station and can export; Admin manages users and devices but cannot edit alerts.
- **Evidence is read-only:** the app's database user has only INSERT and SELECT rights on the alerts table, and a database trigger blocks UPDATE and DELETE. The website re-checks the hash chain every night and shows a tamper badge if it breaks.
- **OWASP Top 10:** parameterised queries (no SQL injection), escaped output plus a Content Security Policy (no XSS), CSRF tokens, schema validation on every input, no error details shown to users.
- **Live updates to the browser:** WebSocket over WSS, opened only after login, carrying only alerts the user's role may see.
- **Location privacy:** the alert map with GPS points is never public; only logged-in roles see it.

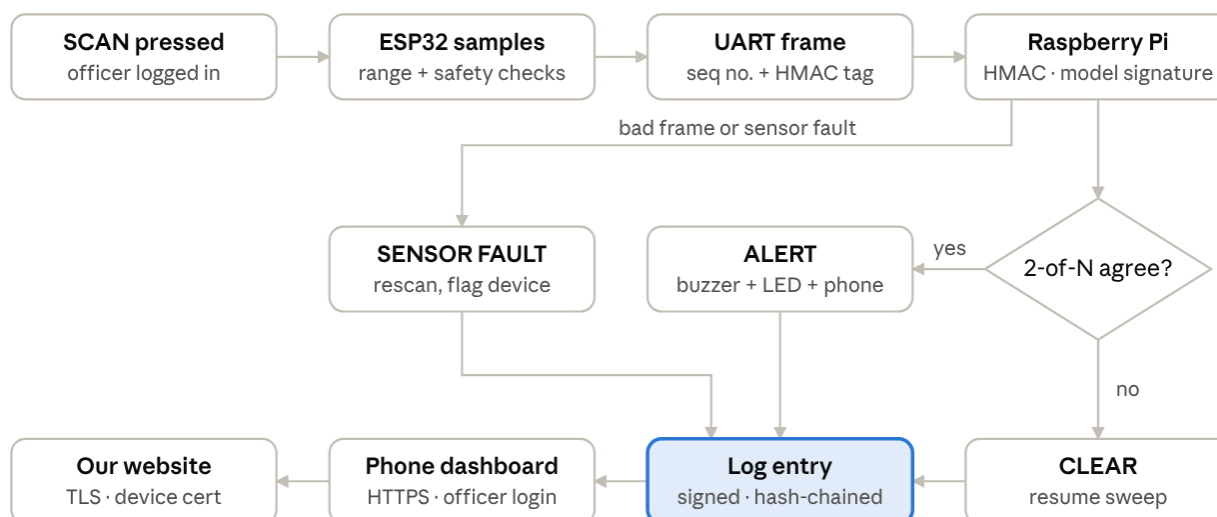
- **Secrets:** database passwords, API keys and the CA key live in environment variables or a secret manager, never in the GitHub repo.
- **Database:** not reachable from the internet, encrypted at rest, with daily encrypted backups.
- **Abuse protection:** rate limits on login and the Device API; a CDN/WAF in front of the site for DDoS protection.
- **Audit trail:** logins, views, exports and user changes are logged and kept for 180 days, in line with CERT-In (section 7).
- **Testing:** an OWASP ZAP scan and dependency check (pip-audit / npm audit) before every release.

Tools: FastAPI or Flask, PostgreSQL, Nginx, Let's Encrypt, pyotp, OWASP ZAP.

5. Secure data flow: from SCAN to evidence

Every scan ends in a signed log entry, whatever the result: ALERT, CLEAR or SENSOR FAULT. The checks from section 4 sit on each hop of the one-button scan cycle, and every failure path is fail-safe: it never turns into a silent CLEAR.

Secure scan-to-alert flow



One-button scan cycle with security checks at each hop

The log entry is highlighted because it is what goes to court; the phone and the website only receive copies of it.

Hop	Check	If the check fails
SCAN pressed	Active officer session	Scan still runs; entry marked "no officer session"
ESP32 samples	Sensor range, stuck-value, heater/pump caps	SENSOR FAULT, heater cut
UART frame	HMAC tag, sequence number	Frame dropped; repeated failures → SENSOR FAULT
Raspberry Pi	Model signature at load; out-of-range input	Last good model used; UNCERTAIN – RESCAN
2-of-N consensus	At least 2 sensor channels agree	Logged as CLEAR with raw readings kept
Log entry	Hash chain + device signature	Write refused; buzzer signals device error
Phone dashboard	HTTPS + login	Alert still shown by buzzer/LED; phone syncs later
Website	Mutual TLS + chain check	Upload rejected; lathi retries, server raises tamper flag

6. Monitoring, logging and incident response

The lathi keeps a second, separate **security event log** so that attacks leave a trace even when detection works normally. It is hash-chained and encrypted exactly like the alert log (4.6).

Events we record

- Boots, shutdowns and watchdog restarts
- Failed and successful officer logins, lockouts, new devices joining the hotspot
- UART frames rejected for a bad HMAC or replayed sequence number
- Model or update packages rejected for a bad signature
- USB maintenance sessions, software updates, log exports
- Clock jumps, lost GPS fix, low battery, sensor faults

The phone dashboard shows a simple health badge (OK / warning / tamper) from these events. The full event log also syncs to our website (4.8), where repeated warnings from one lathi are flagged.

Incident playbook

Incident	Immediate action	Follow-up
Lathi lost or stolen	Officer reports to post supervisor	Revoke device certificate and hotspot passphrase; data stays encrypted (4.6, 4.8)
Hash chain broken	Take lathi out of service; do not reset or reflash	Image the SD card; compare with the copy already stored on the website
Repeated UART HMAC failures	Treat as hardware tampering; check seals	Replace ESP32 board; re-provision keys
Model or update rejected	Lathi keeps running last good version	Reinstall signed package over USB; investigate source
Many failed logins or unknown devices	Change hotspot passphrase and officer PINs	Review event log for the time and location
Suspected website breach	Isolate server; preserve logs	Report to CERT-In within the required window (section 7)

7. Compliance mapping

The controls above line up with four Indian legal frameworks and two design baselines. Exact obligations for an RPF-deployed device should be confirmed with RPF's legal cell in Phase 2.

Framework	What it asks	How Smart Lathi meets it
CERT-In Directions (Apr 2022)	Report cyber incidents within 6 hours; keep ICT system logs securely for 180 days in India; sync clocks to Indian NTP	Security event log (section 6); website server log retention and NTP sync (4.9); incident playbook
DPDP Act 2023 + DPDP Rules 2025	Reasonable security safeguards and breach notification for personal data; Rules notified Nov 2025, fully in force by about May 2027	Officer IDs and GPS trails encrypted (4.6); least-privilege roles (4.5); breach steps in playbook. Law-enforcement exemptions may apply to consent duties, but security duties stay

Framework	What it asks	How Smart Lathi meets it
IT Act 2000	Protection of computer resources; CERT-In's powers come from Section 70B	Hardening (4.3), access control (4.5), audit trail (6)
Bharatiya Sakshya Adhiniyam 2023 (Section 63)	Electronic records need integrity and a certificate to be admissible as evidence	Signed, hash-chained logs, court-ready export with verification script and chain of custody (4.6)
OWASP IoT Top 10 / ETSI EN 303 645	Baselines for connected devices: no default passwords, secure updates, secure storage, minimal attack surface	Covered by 4.1, 4.3, 4.5, 4.6 and 4.7

Worth noting: CERT-In's list of reportable incidents includes attacks on AI/ML systems, so tampering with Model A or B (4.4) is itself a reportable event once the system is in service ([source](#)).

8. Implementation plan and showcase demo

We implement security in two stages. By the showcase, every alert is protected from sensor to website; permanent hardware locks and government infrastructure follow in an RPF pilot.

Control	Stage
Hash-chained, signed alert log (4.6)	By the showcase
Encrypted log database (4.6)	By the showcase; hardware secure element at RPF pilot
Officer phone dashboard: HTTPS, PIN login, roles (4.5)	By the showcase
Hotspot WPA2 + device cap (4.5)	By the showcase
Pi hardening: no default login, firewall, non-root service (4.3)	By the showcase; read-only root at RPF pilot
ESP32 sensor sanity checks + heater safety caps (4.1)	By the showcase

Control	Stage
UART frames with HMAC + sequence number (4.2)	By the showcase
Signed ML model, verified before loading (4.4)	By the showcase
Lathi → website link: TLS, device certificate, signed entries, acks (4.8)	By the showcase
Website: HTTPS, login + 2FA, roles, read-only evidence, OWASP checks (4.9)	By the showcase
Court-ready export + verifier script (4.6)	By the showcase; forensic-lab validation at RPF pilot
Security event log + health badge (6)	Basic by the showcase; full monitoring at RPF pilot
Signed updates (4.7)	Over USB by the showcase; over-the-air at RPF pilot
ESP32 Secure Boot + flash encryption (4.1)	RPF pilot (permanent eFuse change)
Raspberry Pi 4 signed boot (4.3)	RPF pilot (permanent setting)
Government cloud hosting, RPF certificate authority	RPF pilot

What we will demonstrate at the showcase (about 2 minutes)

1. Scan a proxy sample: the buzzer and phone show an ALERT, and the same entry appears live on the website with its hash and signature.
2. Edit one old row in the lathi's database by hand: the verifier pinpoints the broken link and the website shows a tamper badge.
3. From a laptop with no device certificate, try to post a fake alert to the website: the Device API rejects it.
4. Copy an unsigned model file onto the Pi: it is rejected and the lathi keeps running the last good model.
5. Cut the internet and scan again: the alert still fires and is logged, then uploads to the website on reconnect.

Design choices we will finalise before the showcase

- **Internet link:** 4G USB dongle or RPF post Wi-Fi; the security design in 4.8 is the same for both.
- **Device API protocol:** MQTT over TLS for lightweight live updates, or HTTPS REST for simpler integration.